

Task1: Used Car Price Prediction (Regression)

Loading dataset

```
In [5]: import pandas as pd
import numpy as np

df = pd.read_csv("../Data/used_cars.csv")

display(df.head())

print("\nDATA INFO")
df.info()
```

	Price	Make	Model	Year	Mileage	Fuel_Type	Transmission	City	Seller_ID
0	1506276	BMW	320i	2000	149194.0	Diesel	Manual	Karachi	S1520
1	1178741	Toyota	Corolla	2007	269951.0	NaN	Manual	Rawalpindi	S5557
2	3212917	Audi	A3	2022	226571.0	Hybrid	Automatic	Karachi	S6639
3	3093277	Suzuki	Alto	2023	245870.0	NaN	Manual	Peshawar	S4150
4	3483487	Toyota	Corolla	2021	124484.0	Hybrid	Manual	Karachi	S6820



```
DATA INFO
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 504 entries, 0 to 503
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Price           504 non-null    int64
 1   Make            504 non-null    object
 2   Model           504 non-null    object
 3   Year            504 non-null    int64
 4   Mileage         461 non-null    float64
 5   Fuel_Type       383 non-null    object
 6   Transmission    321 non-null    object
 7   City            411 non-null    object
 8   Seller_ID       504 non-null    object
 9   Description     333 non-null    object
dtypes: float64(1), int64(2), object(7)
memory usage: 39.5+ KB
```

Data Cleaning

```
In [6]: # finding duplicated rows
duplicates = df.duplicated().sum()
print(duplicates)
```

4

```
In [7]: # dropping duplicated rows
df.drop_duplicates(inplace=True)
```

```
print(df.duplicated().sum())
```

0

```
In [8]: # cheking in for missing values
missing = df.isnull().sum()
print(missing)
```

```
Price          0
Make           0
Model          0
Year           0
Mileage        42
Fuel_Type      119
Transmission   182
City           93
Seller_ID      0
Description    170
dtype: int64
```

```
In [9]: # imputating missing values
# calculating mean value of mileage column to fill the gaps in it
missing_value = df['Mileage'].mean()
df['Mileage'].fillna(missing_value,inplace=True)

# filling missing values in object type columns
# because mean and median is invalid for cat type so we use mode[0] (the most fr
df['Fuel_Type'].fillna(df['Fuel_Type'].mode()[0],inplace=True)
df['Transmission'].fillna(df['Transmission'].mode()[0],inplace=True)
df['City'].fillna(df['City'].mode()[0],inplace=True)
df['Description'].fillna(df['Description'].mode()[0],inplace=True)

print(df.isnull().sum())
```

```
Price          0
Make           0
Model          0
Year           0
Mileage        0
Fuel_Type      0
Transmission   0
City           0
Seller_ID      0
Description     0
dtype: int64
```

```
In [10]: print("Price Skewness:",df['Price'].skew())
print("Mileage Skewness:",df['Mileage'].skew())
print("Year Skewness:",df['Year'].skew())
```

```
Price Skewness: 6.243541248266824
Mileage Skewness: 0.2357286662909585
Year Skewness: -0.08467474992892977
```

```
In [11]: '''skew value of price is extemely asymetrical which is bad for the model
strategy: use log transformation to compress large values and spread small ones

df['Price'] = np.log1p(df['Price'])
print("Price Skewness:",df['Price'].skew())

# tranformed twice to get the perfect distribution/skewness
```

Price Skewness: 0.9108220901663093

```
In [12]: # dropping seller_id and discription as high-cardinality columns
df.drop(columns=['Seller_ID', 'Description'], inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 500 entries, 0 to 503
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Price            500 non-null    float64
1   Make             500 non-null    object
2   Model            500 non-null    object
3   Year             500 non-null    int64
4   Mileage          500 non-null    float64
5   Fuel_Type        500 non-null    object
6   Transmission     500 non-null    object
7   City             500 non-null    object
dtypes: float64(2), int64(1), object(5)
memory usage: 35.2+ KB
```

Categorical and Numerical Encoding

```
In [13]: X=df

cat_features=X.select_dtypes(include=object).columns # grabs all text/categorical
num_features=X.select_dtypes(exclude=object).columns # grabs all numeric columns

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer # applies diff transformer to diff

ohe=OneHotEncoder() # Converts text categories into numbers
scalar=StandardScaler() # Scales numbers so they're all on same scale. Formula:

# preprocessor returns a numpy array after transformation
preprocessor= ColumnTransformer(
    [
        ("OneHotEncoder", ohe, cat_features),
        ("StandardScaler", scalar, num_features)
    ]
)
```